

Exercise 4 — Introduction to Git and GitHub

Step-by-step guide for first-time Git users (Windows).

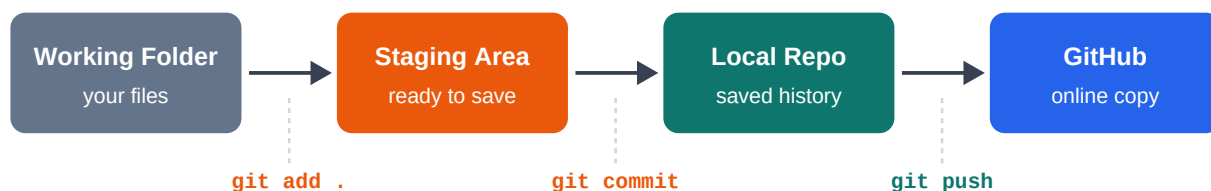
Git vs GitHub — what's the difference?

	Git	GitHub
What	A tool on your computer	A website (github.com)
Job	Saves versions (history) of your code	Stores your code online & shares it
Example	Like "Save As" with full history	Like Google Drive for code

Words You Must Know

Word	Simple meaning
Repository (repo)	A project folder that Git is tracking
Commit	A saved snapshot of your files, with a message
Branch	A separate copy to work on, without disturbing main
Remote (origin)	The GitHub copy of your repo
Push	Send your commits from your computer to GitHub
Merge	Join one branch's changes into another

How Git Moves Your Files



Tip: Use **Git Bash** (installed with Git) for all commands. In the terminal blocks below, **green lines starting with \$** are what you type — don't type the \$. White lines are the output.

a) Setup

AIM

To install Git, configure it with user name and email, and create a GitHub account with a personal access token.

1. Install Git

Step	Do this
1	Go to git-scm.com → click Download for Windows
2	Run the installer → keep clicking Next (default options are fine) → Install
3	Open Git Bash from the Start menu and check the version

● ● ● Git Bash

```
$ git --version
git version 2.xx.x.windows.1
```

2. Configure Git (name & email)

● ● ● Git Bash

```
$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"
$ git config --global init.defaultBranch main
$ git config --list
user.name=Your Name
user.email=you@example.com
init.defaultbranch=main
```

- Use the **same email** as your GitHub account.
- **--global** = set once, works for all projects.
- **init.defaultBranch main** = new repos start on the "main" branch.

3. Create GitHub Account + Personal Access Token (PAT)

Step	Do this
1	Go to github.com → Sign up → enter email, password, username → verify email
2	Click your profile photo (top right) → Settings
3	Scroll down left menu → Developer settings
4	Personal access tokens → Tokens (classic) → Generate new token (classic)
5	Note: " <i>lab token</i> ", Expiration: 30 days , tick the repo checkbox
6	Click Generate token → copy it immediately and save it somewhere safe

Why a token? GitHub does not accept your account password in the terminal. When Git asks for a password during **git push**, paste the **token** instead.

Warning: the token is shown only once — and never share it.

b) Basic Git Workflow

AIM

To create a local repository, add and commit files, connect it to GitHub and push the code.

Step 1 — Create a project folder and start a Git repository (**git init**)

```
● ● ● Git Bash

$ mkdir my-project
$ cd my-project
$ git init
Initialized empty Git repository in C:/Users/You/my-project/.git/
```

Step 2 — Create a file and add it (**git add .**)

```
● ● ● Git Bash

$ echo "Hello Git" > hello.txt
$ git add .
```

Step 3 — Commit the file (**git commit**)

```
● ● ● Git Bash

$ git commit -m "Initial commit"
[main (root-commit) a1b2c3d] Initial commit
1 file changed, 1 insertion(+)
create mode 100644 hello.txt
```

Step 4 — Create an **empty** repo on GitHub

github.com → + → **New repository** → name **my-project** → **don't** tick README → **Create repository**

Step 5 — Connect to GitHub (**git remote add origin**) — replace *username* with yours

```
● ● ● Git Bash

$ git remote add origin https://github.com/username/my-project.git
$ git remote -v
origin https://github.com/username/my-project.git (fetch)
origin https://github.com/username/my-project.git (push)
```

Step 6 — Push to GitHub (**git push**)

```
● ● ● Git Bash

$ git push -u origin main
Enumerating objects: 3, done.
Writing objects: 100% (3/3), 225 bytes | 225.00 KiB/s, done.
To https://github.com/username/my-project.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Login popup? Windows may open a GitHub sign-in window — just sign in. If it asks for a **password** in the terminal, paste your **token**.

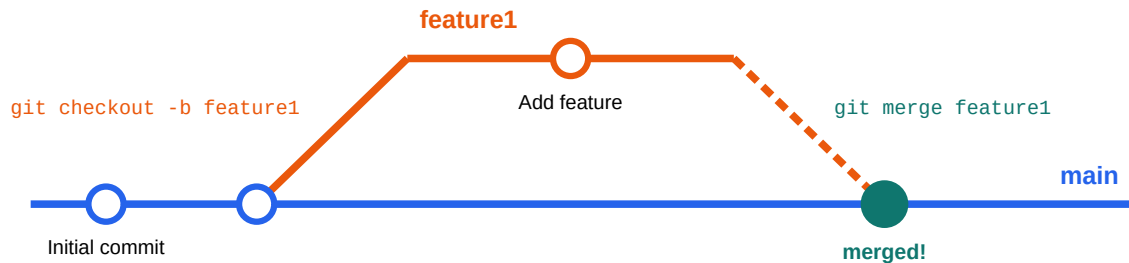
Output check: refresh your repo page on GitHub — **hello.txt** is now there!

c) Branching and Collaboration

AIM

To create a branch, merge it into main, and resolve a merge conflict.

WHAT WE ARE DOING



Step 1 — Create a new branch and switch to it

```
● ● ● Git Bash

$ git checkout -b feature1
Switched to a new branch 'feature1'
```

Step 2 — Add something new on this branch and commit it

```
● ● ● Git Bash

$ echo "New feature" > feature.txt
$ git add .
$ git commit -m "Add feature"
[feature1 e4f5a6b] Add feature
1 file changed, 1 insertion(+)
```

Step 3 — Go back to main and merge feature1

```
● ● ● Git Bash

$ git checkout main
Switched to branch 'main'
$ git merge feature1
Updating a1b2c3d..e4f5a6b
Fast-forward
 feature.txt | 1 +
1 file changed, 1 insertion(+)
$ git push
```

• **-b** means "create a new branch". • You must be **on main** before merging into main. • **Fast-forward** = simple merge, no conflict.

c) Resolving a Merge Conflict (Guided)

A **conflict** happens when **two branches change the same line** of the same file. Git cannot decide which one to keep — so **you** decide.

Step 1 — Create the conflict on purpose — change **hello.txt** in two branches

```
● ● ● Git Bash

$ git checkout -b feature2
$ echo "Hello from feature2" > hello.txt
$ git add .
$ git commit -m "Change in feature2"

$ git checkout main
$ echo "Hello from main" > hello.txt
$ git add .
$ git commit -m "Change in main"
```

Step 2 — Try to merge — Git reports a conflict

```
● ● ● Git Bash

$ git merge feature2
Auto-merging hello.txt
CONFLICT (content): Merge conflict in hello.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Step 3 — Open **hello.txt** (Notepad or VS Code). Git has marked both versions:

<<<<<< HEAD	← start of conflict
Hello from main	← version in main (your current branch)
=====	← divider
Hello from feature2	← version in feature2
>>>>>> feature2	← end of conflict

Step 4 — Fix it: keep what you want and **delete all 3 marker lines** (<<<, ==, >>>). Save the file:

```
Hello from main and feature2
```

Step 5 — Tell Git the conflict is fixed, then commit

```
● ● ● Git Bash

$ git add hello.txt
$ git commit -m "Resolve merge conflict"
[main 9c8d7e6] Resolve merge conflict
$ git push
```

KEY POINTS

- Conflict = same line changed differently in two branches.
- Everything between <<<<<< and >>>>>> is the fight zone — pick, combine, or rewrite.
- Always **git add** + **git commit** after fixing.

Command Cheat Sheet

Command	What it does
<code>git --version</code>	Check Git is installed
<code>git config --global user.name "..."</code>	Set your name
<code>git init</code>	Start tracking a folder
<code>git status</code>	See what changed
<code>git add .</code>	Stage all changes
<code>git commit -m "msg"</code>	Save a snapshot
<code>git remote add origin URL</code>	Link to GitHub
<code>git push -u origin main</code>	Upload to GitHub (first time)
<code>git push</code>	Upload again (after first time)
<code>git checkout -b name</code>	Create + switch to a new branch
<code>git checkout main</code>	Switch back to main
<code>git merge name</code>	Merge branch into current branch
<code>git log --oneline</code>	See commit history

RESULT

Git was installed and configured, a GitHub account and personal access token were created, a local repository was pushed to GitHub, and branching, merging and merge-conflict resolution were performed successfully.